

Lecture: 1000

SRAM vs DRAM & Architectures

**Why Computers Have So Many Kinds of
Memory**

From a 6-transistor cell to the memory wall — the trade-offs that shape every computer.

CSA222: Modern Computer Architecture

Register Files Are Tiny

L7's register file holds 32 words. Real programs need billions. We need new memory.



Register file

32–128 words of flip-flops
— blazing fast, right by
the ALU (L7).



But it's small

Flip-flops are expensive:
20+ transistors per bit.
Can't scale to GBs.



Programs need more

A photo is millions of
bytes; an OS is billions.
Registers can't hold that.



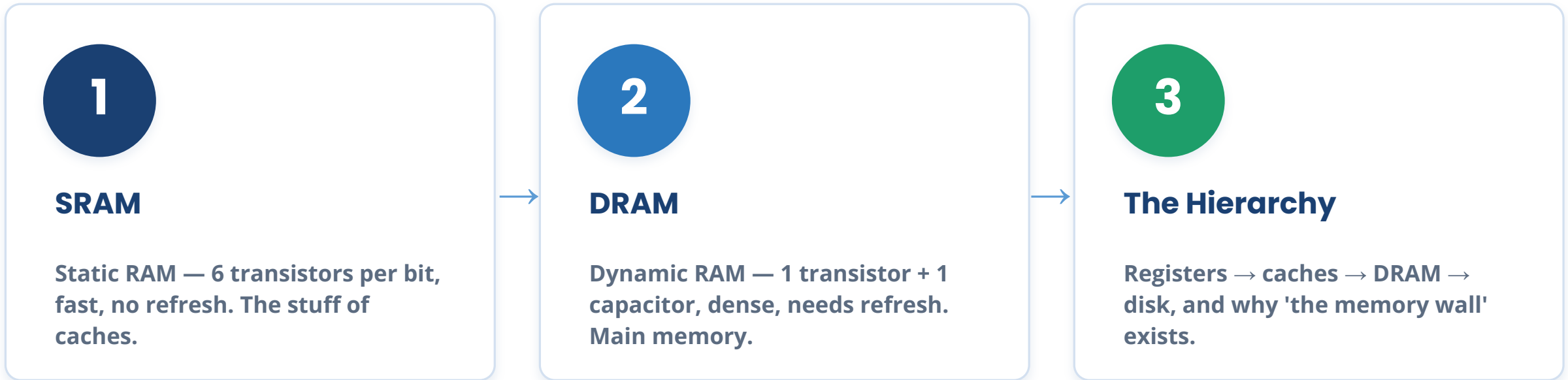
Different jobs, different memory

So we use several memory
technologies, each tuned
for its role.

The core tension: fast memory is small and expensive; big memory is slow and cheap. Today we see why.

Today's Roadmap

Two memory technologies, then how they stack into a hierarchy.



The punchline: no single memory is fast, big, AND cheap. Computers cheat by layering them — the memory hierarchy.

1. SRAM

Static RAM: a bit stored in cross-coupled inverters. Fast and stable — but it costs six transistors per bit.

What Is SRAM?

'Static' because it holds its value with no refresh — as long as the power stays on.

THE IDEA

SRAM stores each bit in a bistable latch — two cross-coupled inverters, exactly like the one from L5.

Once set, the feedback loop holds the value indefinitely.
No clock, no refresh — the bit just stays.

*That stability is why it's called **STATIC RAM**.*

AT A GLANCE

6 transistors

per bit cell

A few ns

access time — very fast

No refresh

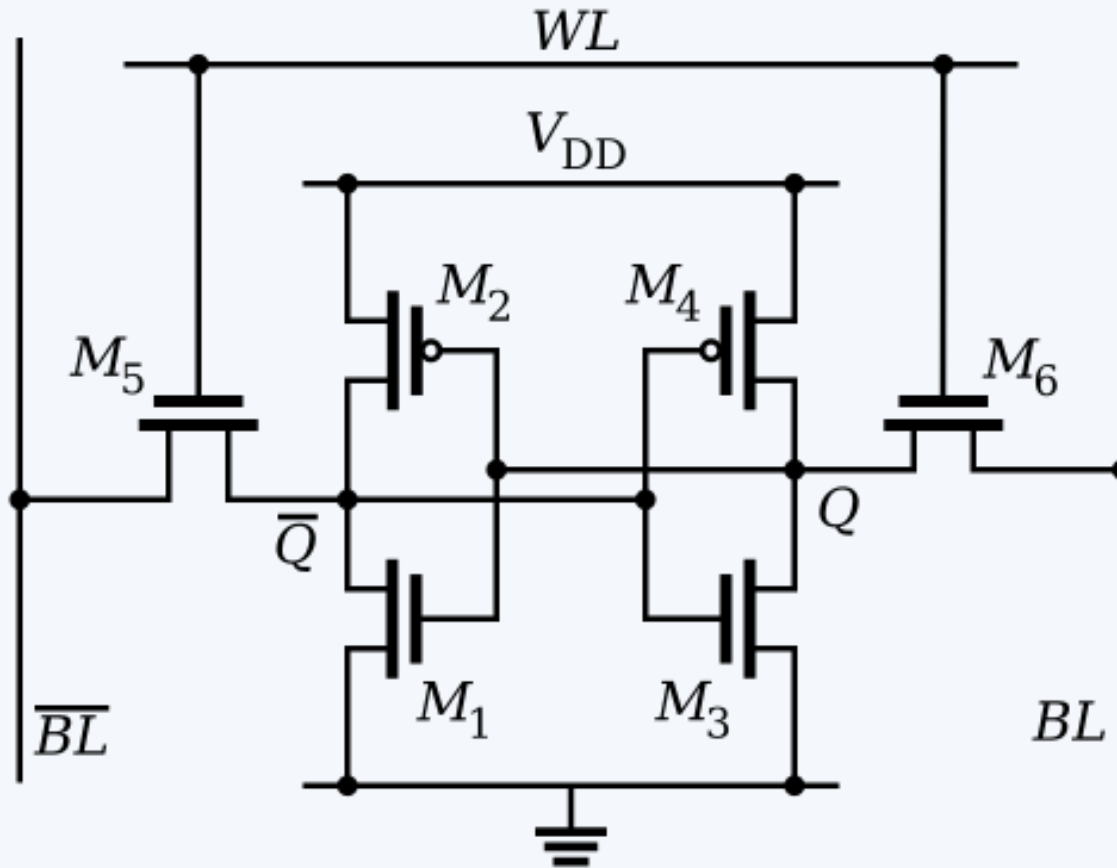
holds while powered

Expensive

large area per bit

The 6T SRAM Cell

Four transistors form two cross-coupled inverters; two more are access gates.



4 transistors (2 inverters) + 2 access = 6T

THE PARTS

Cross-coupled inverters (4T): store the bit, exactly the L5 bistable.

Access transistors M5, M6 (2T): connect the cell to the bit lines when the word line is high.

Word line = 'select this cell'. Bit lines = read/write the data (and its complement).

How SRAM Holds a Bit

The feedback loop reinforces itself — the bit is 'self-healing' while powered.

SELF-REINFORCING

If $Q = 1$, the first inverter drives $Q' = 0$, and the second inverter drives Q back to 1.

The loop constantly 'refreshes itself' from its own power supply. Tiny leakage is instantly corrected.

So the value is rock-solid — no external refresh circuit needed, ever.

Trade-off: this stability costs 6 transistors per bit — a lot of silicon for one bit.

VS A CAPACITOR

SRAM stores the bit **ACTIVELY** — powered transistors constantly hold it.

DRAM (next section) stores it **PASSIVELY** — as charge on a capacitor that slowly leaks away.

Active storage = fast & stable but bulky. Passive storage = tiny & cheap but needs constant refreshing. That single choice explains almost every difference between them.

Reading & Writing SRAM

Raise the word line to connect the cell; the bit lines do the rest.

READ

- 1 Pre-charge both bit lines high.
- 2 Raise the word line → cell connects.
- 3 The stored bit pulls one bit line low.
- 4 A sense amplifier detects which side dropped.

WRITE

- 1 Drive the bit lines to the desired value.
- 2 Raise the word line → cell connects.
- 3 The strong bit-line drivers overpower the cell.
- 4 The latch flips to the new value and holds it.

Reads are non-destructive — the cell keeps its value. That's another SRAM advantage over DRAM.

SRAM: The Scorecard

Fast and simple to use — but the six transistors cost you.



Very fast

Access in a few nanoseconds — no refresh delay, non-destructive reads.



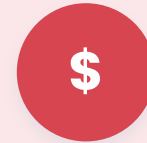
No refresh

The latch holds itself. Simpler controller, lower access latency.



Low density

6 transistors per bit → far fewer bits per mm² than DRAM.



Expensive & power-hungry

More silicon per bit and constant static power. Costly at scale.

Perfect for small, speed-critical memory → this is exactly why caches (L1/L2/L3) are built from SRAM.

2. DRAM

Dynamic RAM: one transistor, one capacitor. Tiny and cheap — but the charge leaks, so it must be refreshed.

What Is DRAM?

'Dynamic' because the stored charge leaks away and must be constantly refreshed.

THE IDEA

DRAM stores each bit as **CHARGE** on a tiny capacitor:
charged = 1, empty = 0.

One transistor acts as a switch to access that capacitor.
That's it — just 1T + 1C per bit.

*But capacitors leak. Within milliseconds the charge fades —
so DRAM must be refreshed thousands of times a second.
Hence 'DYNAMIC'.*

AT A GLANCE

1T + 1C

per bit cell — tiny

50–100 ns

access — slower than SRAM

Needs refresh

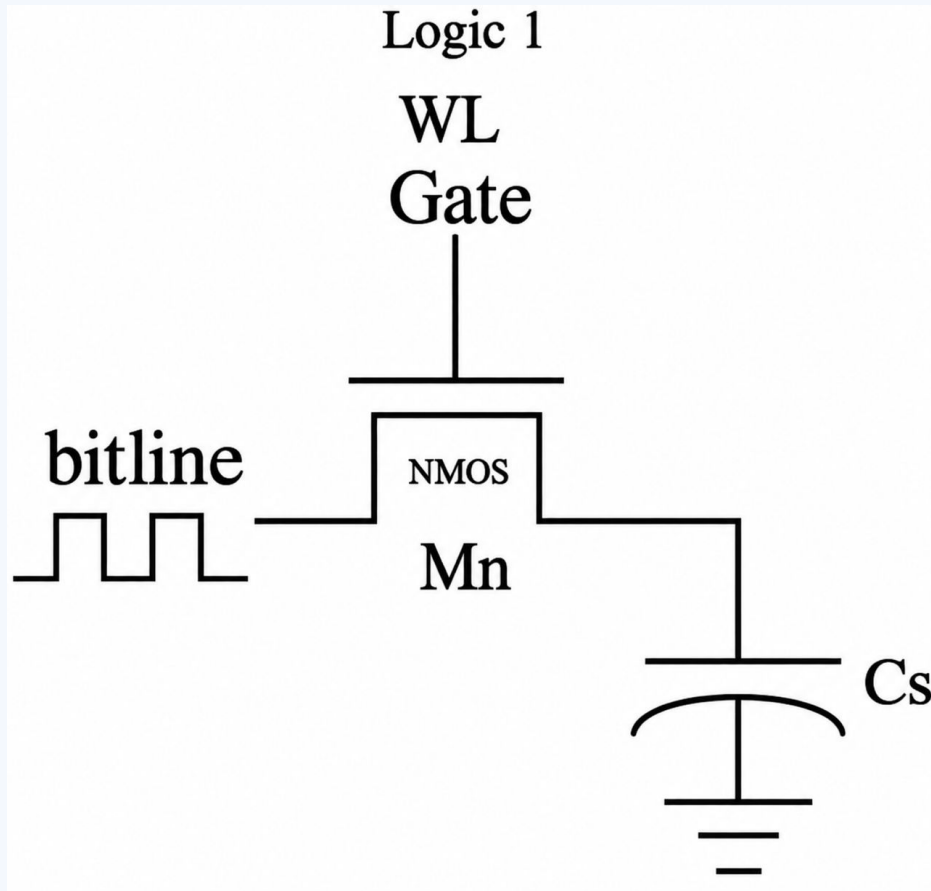
every few milliseconds

Very dense

cheap per bit at scale

The 1T1C DRAM Cell

One access transistor, one storage capacitor — the smallest memory cell there is.



Just 1 transistor + 1 capacitor = 1T1C

HOW IT WORKS

Word line high → transistor opens → capacitor connects to the bit line.

Write: drive the bit line to charge or drain the capacitor.

Read: sense the tiny charge the capacitor dumps onto the bit line.

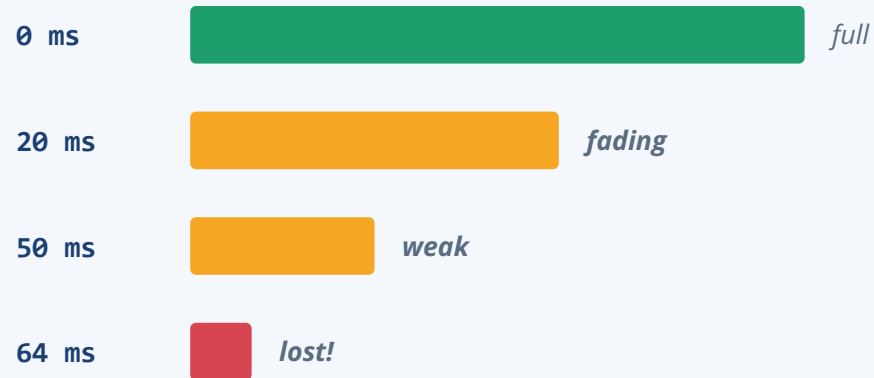
One capacitor vs six transistors is why DRAM packs ~6× more bits into the same area.

Why DRAM Needs Refresh

The capacitor leaks. Left alone, a 1 fades to 0 within milliseconds.

THE LEAK

Charge on the capacitor over time:



So DRAM controllers read and rewrite every row every ~64 ms — that's REFRESH.

THE COST OF REFRESH

A dedicated refresh controller cycles through every row, thousands of times per second.

During refresh, that part of memory is briefly unavailable — a small performance and power cost.

SRAM never pays this cost — its latch refreshes itself from power. This is the single biggest practical difference between the two.

Destructive Read & Sense Amplifiers

Reading a DRAM cell drains its capacitor — so the value must be written straight back.

1

Access

Word line opens the transistor; the capacitor shares its charge with the bit line.

2

Tiny signal

The charge is minuscule — it barely nudges the bit line's voltage.

3

Sense amp

A sense amplifier detects that tiny nudge and swings it to a full 0 or 1.

4

Write-back

The read drained the cell — so the amplifier immediately rewrites the value back.

WHY IT'S SLOWER

Every read is really a read + a rewrite.

The sense amplifier needs time to resolve a tiny charge into a clean bit.

Add row activation and precharge, and you get DRAM's ~50–100 ns latency — over 10× slower than SRAM.

DRAM: The Scorecard

Dense and cheap — the reasons it's used for gigabytes of main memory.



Very dense

1T1C packs ~6× more bits per mm² than SRAM. Gigabytes fit on one chip.



Cheap per bit

Fewer components → far lower cost at scale. Ideal for bulk memory.



Slower

50–100 ns access, destructive reads, row activation overhead.



Needs refresh

Constant refresh circuitry costs power and briefly stalls access.

Perfect for large, cost-sensitive memory → this is exactly why main memory (your 8/16/32 GB RAM) is DRAM.

Quick Check

Show of hands — the defining difference.

Q. Which memory needs a periodic refresh to keep its data — and why?

A

SRAM — latch leaks

B

DRAM — capacitor
leaks

✓ *charge fades in ms*

C

Both equally

D

Neither

DRAM's capacitor loses charge and must be refreshed. SRAM's cross-coupled latch holds itself from power — no refresh.

3. SRAM vs DRAM

Same job — store bits — but opposite trade-offs. Each wins in a different role.

SRAM vs DRAM: Side by Side

The comparison you need cold for exams and interviews.

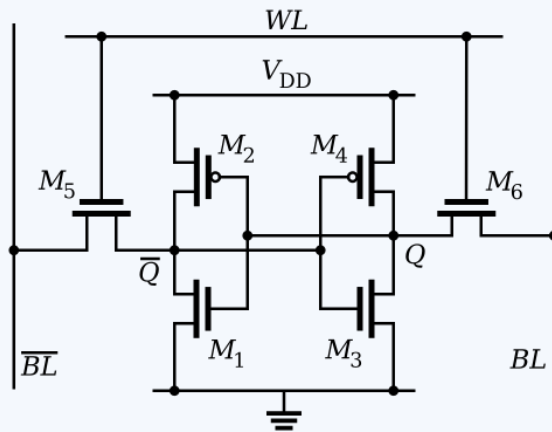
	SRAM	DRAM
Cell	6 transistors	1 transistor + 1 capacitor
Density	Low	High (~6× SRAM)
Speed	Fast (a few ns)	Slower (50–100 ns)
Refresh	Not needed	Every few ms
Read	Non-destructive	Destructive (write-back)
Cost / bit	High	Low
Used for	Caches (L1/L2/L3)	Main memory (RAM)

One sentence: SRAM trades density for speed; DRAM trades speed for density. Neither is 'better' — they're different tools.

Why DRAM Wins on Density

Same silicon area, very different bit counts — it's a transistor-count story.

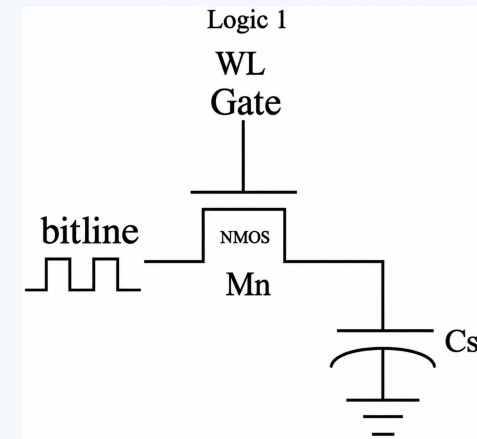
SRAM — 1 BIT



6 transistors → 1 bit

bulky, but fast & stable

DRAM — 1 BIT



1 transistor + 1 capacitor → 1 bit

tiny → ~6× more bits per mm²

Where Each One Lives

The trade-offs decide the role: SRAM close and fast, DRAM far and big.



SRAM → Caches

L1, L2, L3 caches sit on the CPU die, holding KBs–MBs of the hottest data. Speed is everything here.



DRAM → Main memory

The 8/16/32 GB 'RAM' sticks on your motherboard. Capacity and cost per bit matter most.



SRAM → Register files & buffers

Tiny, ultra-fast on-chip structures — like the register file from L7 — use SRAM-style cells.



DRAM → Frame buffers, GPUs

Graphics memory (GDDR) and bulk buffers are DRAM variants — huge capacity at reasonable cost.

From Gates to a Complete Machine

A computer is more than an ALU — it needs memory, I/O, and a way to wire them together.



The question of computer organisation: how exactly do these connect — and how many buses?

The answer shapes everything: performance, cost, and how you program the machine. Two classic blueprints follow.

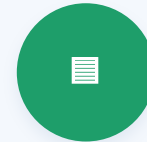
The Key Insight: Programs ARE Data

Because code lives in memory as numbers, it can be treated like any other data.



Compilers & assemblers

A program that writes another program — it outputs instructions as data into memory.



Loading & the OS

The OS loads a program by copying its instruction-bytes into memory, then jumps to them.



Just-in-time compilation

JITs (JVM, browsers) generate machine code at runtime and then execute it — data becomes code.



Self-modifying code

Programs can, in principle, rewrite their own instructions — powerful, and a security minefield.

This blurring of code and data is von Neumann's superpower — and the root of whole classes of security exploits.